

Introduction to Data Science — Midsem Answer Key

Part A — Very Short Answer (2 Marks Each)

Q1. What are different tasks performed during data cleaning?

Data cleaning involves the following tasks:

- 1. **Handling Missing Values** — Identifying and filling (mean/median/mode imputation) or removing records with missing entries.
- 2. **Removing Duplicates** — Detecting and eliminating duplicate records from the dataset.
- 3. **Noise Removal** — Smoothing noisy data using techniques like binning, regression, or clustering.
- 4. **Resolving Inconsistencies** — Correcting inconsistent data formats, naming conventions, or coding schemes (e.g., "M" vs "Male").
- 5. **Outlier Detection** — Identifying data points that deviate significantly from the rest and deciding whether to keep or remove them.

Q2. Differentiate between primary and secondary data, with suitable example.

Aspect	Primary Data	Secondary Data
Definition	Data collected first-hand by the researcher for a specific purpose.	Data that has already been collected by someone else for a different purpose.
Source	Surveys, experiments, interviews, observations.	Government reports, journals, databases, web data.
Cost	Higher (requires effort to collect).	Lower (readily available).
Example	A company conducting a customer satisfaction survey.	Using Census data published by the government for market analysis.

Q3. Write a Python code to add new column in the existing DataFrame.

```
import pandas as pd

# Create a sample DataFrame
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})

# Method 1: Direct assignment
df['City'] = ['Delhi', 'Mumbai', 'Pune']

# Method 2: Using assign()
df = df.assign(Salary=[50000, 60000, 70000])

print(df)
```

Output:

	Name	Age	City	Salary
0	Alice	25	Delhi	50000
1	Bob	30	Mumbai	60000
2	Charlie	35	Pune	70000

Q4. Explain forward fill and backward fill with suitable example.

Forward Fill (ffill): Fills a missing value with the **previous** (last valid) value in the column.

Backward Fill (bfill): Fills a missing value with the **next** valid value in the column.

Example:

Index	Value (Original)	After ffill	After bfill
0	10	10	10
1	NaN	10	30
2	30	30	30
3	NaN	30	50
4	50	50	50

```
import pandas as pd
s = pd.Series([10, None, 30, None, 50])
print(s.ffill()) # Forward fill
print(s.bfill()) # Backward fill
```

Q5. Outline difference between structured and unstructured data with suitable example.

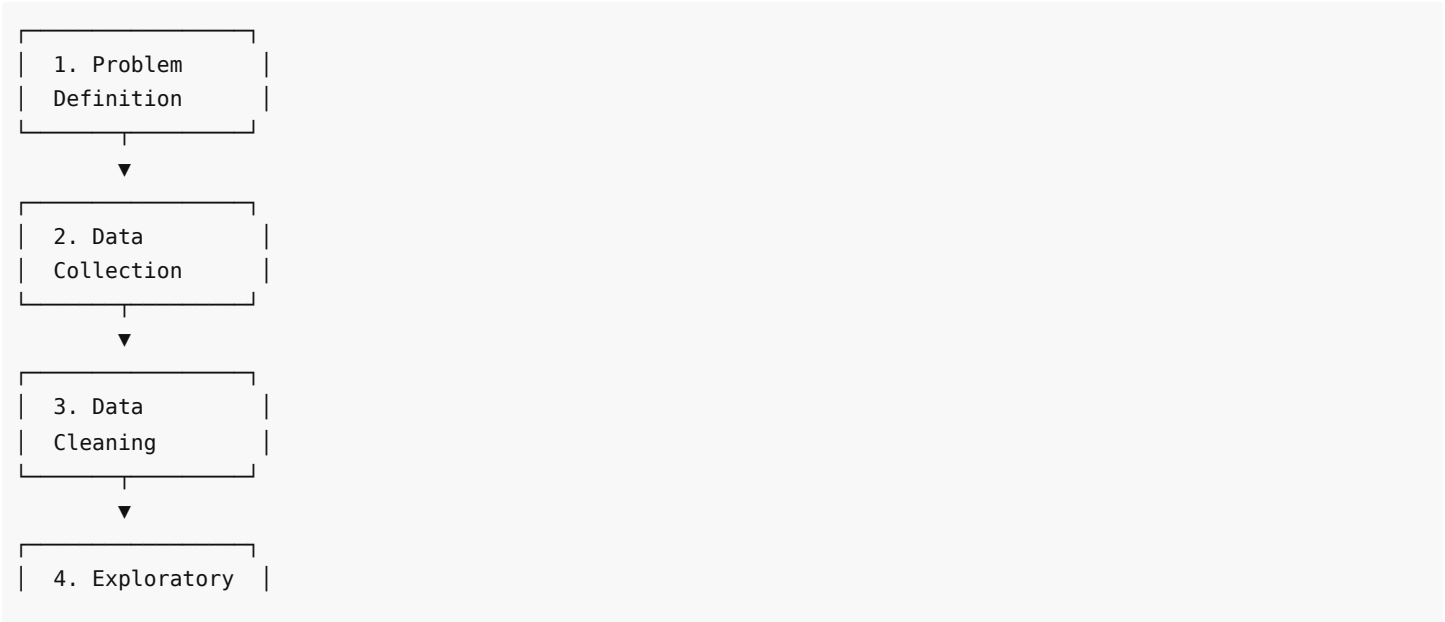
Aspect	Structured Data	Unstructured Data
Definition	Data organized in a fixed, predefined format (rows & columns).	Data with no predefined structure or format.
Storage	Relational databases (SQL tables).	NoSQL databases, data lakes, file systems.
Ease of Analysis	Easy to search, query, and analyse.	Difficult to process and analyse without specialized tools.
Example	An Excel spreadsheet of student marks with columns: Roll No, Name, Marks.	Emails, social media posts, images, videos, audio files.

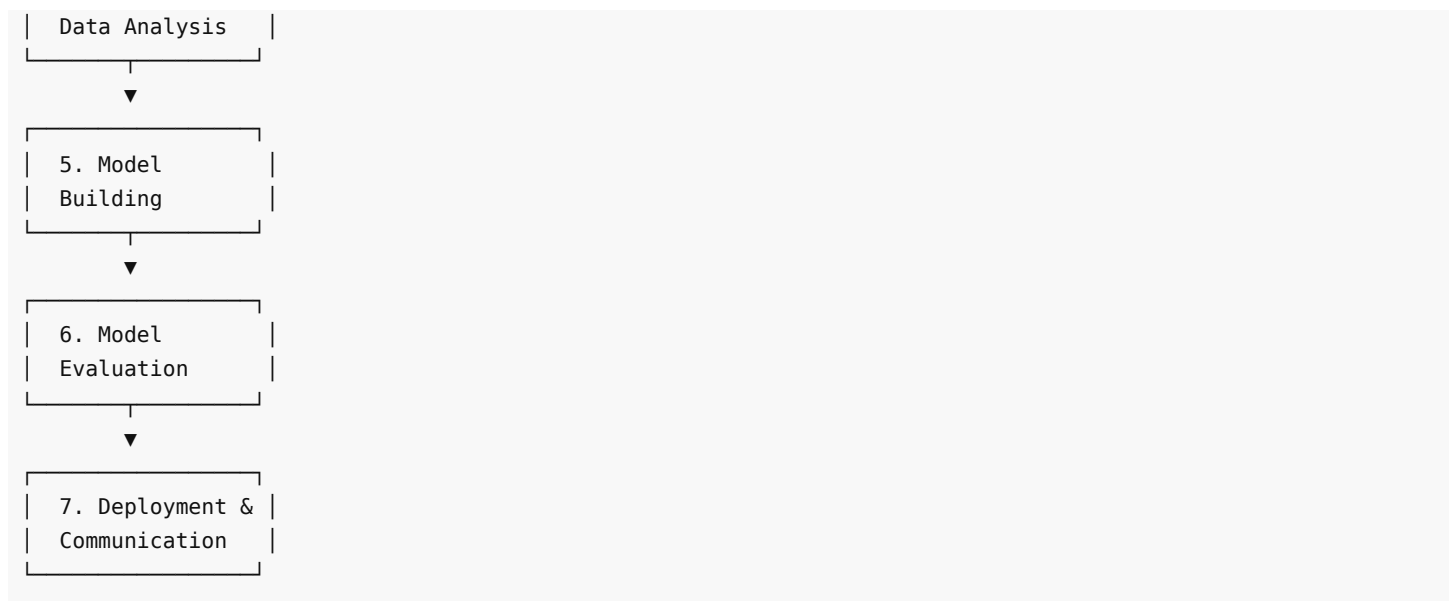
Part B — Short Answer (5 Marks Each, Do Any 4)

Q6. Define data science. As per your understanding draw the life cycle of the data science project.

Data Science is an interdisciplinary field that uses scientific methods, statistical techniques, algorithms, and computational systems to extract meaningful knowledge and insights from structured and unstructured data.

Data Science Lifecycle:





Phases Explained:

1. **Problem Definition** — Clearly define the business question or objective.
2. **Data Collection** — Gather data from various sources (databases, APIs, surveys).
3. **Data Cleaning** — Handle missing values, remove duplicates, fix inconsistencies.
4. **EDA (Exploratory Data Analysis)** — Visualize and summarize data to find patterns using plots and statistics.
5. **Model Building** — Apply machine learning or statistical models to the data.
6. **Model Evaluation** — Assess performance using metrics (accuracy, RMSE, F1-score).
7. **Deployment & Communication** — Deploy the model and present findings to stakeholders.

Q7. Predict the output of the code and justify using operator precedence.

```
x = 7
y = 3
y = x // y          # y = 7 // 3 = 2
z = x**y // y + x % y * 3
# Step-by-step:
# x**y = 7**2 = 49      (Exponentiation – highest precedence)
# x % y = 7 % 2 = 1     (Modulus)
# 49 // 2 = 24          (Floor division – left to right with *)
# 1 * 3 = 3             (Multiplication)
# 24 + 3 = 27           (Addition)
print(z) # Output: 27
```

Output: 27

Justification — Operator Precedence (highest to lowest):

1. ****** (Exponentiation) → $7^{**}2 = 49$
2. **// , % , *** (same level, left to right):
 - $49 // 2 = 24$
 - $7 \% 2 = 1$
 - $1 * 3 = 3$
3. **+** (Addition) → $24 + 3 = 27$

Q8. Analyse the impact of missing data on the accuracy of a data model. Techniques to handle inconsistencies.

Impact of Missing Data:

1. **Reduced Sample Size** — Deletion of rows with missing data reduces the dataset, leading to loss of information.
2. **Biased Results** — If data is not missing at random, models may learn biased patterns.
3. **Reduced Model Accuracy** — Gaps in features lead to poor generalization and lower prediction power.

4. **Invalid Statistical Inferences** — Measures like mean and variance get distorted.

Techniques to Handle Missing Data:

Technique	Description
Deletion	Remove rows (listwise) or columns with missing values. Suitable when missing % is small.
Mean / Median / Mode Imputation	Replace missing values with the mean (numeric), median (skewed), or mode (categorical).
Forward / Backward Fill	Use the previous (ffill) or next (bfill) valid value. Good for time-series data.
Interpolation	Estimate missing values based on surrounding data points (linear, polynomial).
KNN Imputation	Use k-NN to find similar records and fill missing values from their averages.
Predictive Modelling	Train a model (e.g., regression) on complete cases to predict missing values.

Q9. How Systematic and Stratified sampling can be used to check sensor accuracy.

Systematic Sampling:

- Select every *k*-th sensor from the list of 10,000 sensors.
- If we want to check 500 sensors: $k = 10,000 / 500 = \mathbf{20}$.
- Start from a random sensor (e.g., sensor #7), then check sensor 7, 27, 47, 67, ... and so on.
- **Advantage:** Simple to implement, ensures sensors are spread across the entire list.
- **Limitation:** If sensors are arranged in a pattern (e.g., by zone), systematic sampling might miss some zones.

Stratified Sampling:

- Divide the 10,000 sensors into **strata** (groups) based on meaningful criteria like **zone** (industrial, residential, commercial) or **sensor age**.
- From each stratum, randomly select a proportional number of sensors.
- Example: If 3,000 sensors are in industrial zones, 5,000 in residential, and 2,000 in commercial → sample 150, 250, and 100 respectively (for a total of 500).
- **Advantage:** Ensures all zones are represented, reducing bias.
- **Better than systematic** when zones have different environmental conditions.

Conclusion: Stratified sampling is more suitable for this scenario as pollution levels vary significantly across zones, and each zone must be adequately represented.

Q10. What is a CSV file? How is it different from JSON/XML? Code snippet.

CSV (Comma-Separated Values): A plain text file format where each line represents a data record and fields within a record are separated by commas.

Comparison:

Feature	CSV	JSON	XML
Structure	Flat, tabular (rows & columns)	Key-value pairs, nested	Tag-based, hierarchical
Readability	Easy for tabular data	Easy for humans & machines	Verbose but structured
File Size	Smallest	Medium	Largest
Nested Data	Not supported	Supported	Supported
Usage	Spreadsheets, databases	APIs, web apps	Configuration, data exchange

Python Code to Read a CSV File:

```
import pandas as pd

# Reading a CSV file
```

```
df = pd.read_csv('data.csv')

# Display first 5 rows
print(df.head())

# Display shape
print("Shape:", df.shape)

# Display column names
print("Columns:", df.columns.tolist())
```

Part C — Long Answer (10 Marks Each, Do Any 2)

Q11. Explain the following in minimum length:

(i) Write a function that takes a list of numbers and returns the sum.

```
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total

# Or using built-in
def sum_list(numbers):
    return sum(numbers)

print(sum_list([1, 2, 3, 4, 5])) # Output: 15
```

(ii) How do you reverse a string in Python?

```
# Method 1: Slicing
s = "Hello"
reversed_s = s[::-1]
print(reversed_s) # Output: "olleH"

# Method 2: Using reversed() and join()
reversed_s = ''.join(reversed(s))
print(reversed_s) # Output: "olleH"
```

(iii) What is the difference between a tuple and a list?

Feature	List	Tuple
Syntax	[1, 2, 3]	(1, 2, 3)
Mutability	Mutable (can be modified)	Immutable (cannot be modified)
Performance	Slower	Faster (due to immutability)
Methods	More built-in methods (append, insert, remove)	Fewer methods (count, index)
Use Case	When data needs to change	When data should remain constant

(iv) How can you create a dictionary in Python?

```
# Method 1: Curly braces
student = {'name': 'Rajat', 'age': 20, 'course': 'Data Science'}
```

```
# Method 2: dict() constructor
student = dict(name='Rajat', age=20, course='Data Science')

# Method 3: From list of tuples
student = dict([('name', 'Rajat'), ('age', 20), ('course', 'Data Science')])

# Accessing values
print(student['name']) # Output: Rajat
print(student.get('age')) # Output: 20
```

(v) How can you remove duplicates from a list?

```
my_list = [1, 2, 2, 3, 4, 4, 5]

# Method 1: Using set (does not preserve order in older Python)
unique_list = list(set(my_list))
print(unique_list) # Output: [1, 2, 3, 4, 5]

# Method 2: Using dict.fromkeys() (preserves order)
unique_list = list(dict.fromkeys(my_list))
print(unique_list) # Output: [1, 2, 3, 4, 5]

# Method 3: Manual loop
unique_list = []
for item in my_list:
    if item not in unique_list:
        unique_list.append(item)
print(unique_list) # Output: [1, 2, 3, 4, 5]
```

Q12. Critical analysis of Big Data, its features, and its impact. Foundation of Data Science.

Introduction: The exponential growth of digital information from social media, IoT devices, e-commerce, healthcare, and other domains has given rise to **Big Data** — datasets so large and complex that traditional data processing tools are inadequate.

Defining Features of Big Data (5 V's):

V	Description	Example
Volume	Massive amount of data generated.	Facebook generates ~4 petabytes of data daily.
Velocity	Speed at which data is generated and processed.	Stock market data streaming in real-time.
Variety	Different types/formats of data.	Text, images, videos, sensor data, logs.
Veracity	Uncertainty and quality of data.	Social media data may contain misinformation.
Value	Usefulness of the data after processing.	Customer purchase patterns for targeted marketing.

Impact on Contemporary Industries:

- 1. **Healthcare** — Predictive analytics for disease diagnosis, drug discovery, and patient monitoring using wearable devices.
- 2. **E-commerce** — Amazon and Flipkart use recommendation engines powered by Big Data to suggest products.
- 3. **Finance** — Fraud detection systems analyse millions of transactions in real-time to flag suspicious activity.
- 4. **Transportation** — Uber uses Big Data for dynamic pricing, route optimization, and demand prediction.
- 5. **Agriculture** — Precision farming uses satellite and sensor data to optimize irrigation and crop yield.

Big Data as the Foundation of Data Science:

Data Science leverages Big Data through:

- 1. **Data Collection** — Gathering data from diverse sources (APIs, web scraping, IoT sensors).

2. **Storage & Processing** — Using tools like Hadoop, Spark, and cloud platforms to handle massive datasets.
3. **Analysis** — Applying statistical and ML techniques (regression, classification, clustering) to extract patterns.
4. **Visualization** — Using Matplotlib, Tableau, and Power BI to present findings.
5. **Decision Making** — Translating insights into actionable business strategies.

Case-Based Examples:

- **Netflix** — Analyses viewing history of 200M+ users using Big Data to recommend shows and produce original content (e.g., House of Cards was produced based on data-driven insights).
- **COVID-19 Tracking** — Governments worldwide used Big Data from testing, hospitals, and mobility data to track virus spread, allocate resources, and plan vaccination drives.

Conclusion: Big Data has transformed how industries operate by enabling data-driven decision-making. Data scientists play a crucial role in converting raw, large-scale data into meaningful knowledge using analytical tools and computational techniques.

Q13. Python program using Pandas and NumPy for hospital_records.csv.

```
import pandas as pd
import numpy as np

# Read the CSV file
df = pd.read_csv('hospital_records.csv')

# -----
# Task 1: Display the first and last 10 patient names
# -----
print("First 10 Patient Names:")
print(df['Name'].head(10))

print("\nLast 10 Patient Names:")
print(df['Name'].tail(10))

# -----
# Task 2: Total number of missing values in Blood_Pressure and Sugar_Level
# -----
bp_missing = df['Blood_Pressure'].isnull().sum()
sl_missing = df['Sugar_Level'].isnull().sum()
print(f"\nMissing values in Blood_Pressure: {bp_missing}")
print(f"Missing values in Sugar_Level: {sl_missing}")
print(f"Total missing values: {bp_missing + sl_missing}")

# -----
# Task 3: Remove records where Treatment_Cost is missing
# -----
df = df.dropna(subset=['Treatment_Cost'])
print(f"\nRecords after removing missing Treatment_Cost: {len(df)}")

# -----
# Task 4: Fill missing values using mean
# -----
df['Blood_Pressure'] = df['Blood_Pressure'].fillna(df['Blood_Pressure'].mean())
df['Sugar_Level'] = df['Sugar_Level'].fillna(df['Sugar_Level'].mean())
print("\nMissing values after filling:")
print(df[['Blood_Pressure', 'Sugar_Level']].isnull().sum())

# -----
# Task 5: Save the cleaned dataset
# -----
df.to_csv('hospital_cleaned.csv', index=False)
print("\nCleaned dataset saved as 'hospital_cleaned.csv'")
```

Explanation of Key Concepts Used:

- `head(n)` / `tail(n)` — Display first/last n rows.
- `isnull().sum()` — Count missing values per column.
- `dropna(subset=[...])` — Drop rows where specific columns have NaN.
- `fillna(value)` — Replace NaN with a specified value (mean in this case).
- `to_csv()` — Export DataFrame to a CSV file.

— End of Midsem Answer Key —